

Instituto Infnet

Disciplina de Sistemas Embarcados

Sistema Digital de Controle de Acesso

Assessment Final - DR1

Aluno: Renato Noronha Hack
Plataforma utilizada: Tang Nano 9K
Data: 5 de abril de 2026

Este relatório apresenta a modelagem lógica, a implementação em Verilog HDL, a validação por simulação e a validação física em FPGA de um sistema digital de controle de acesso com código de 4 bits, contador síncrono de tentativas e saídas indicadas por LEDs.

Sumário

1	Introdução	2
2	Objetivo do projeto	2
3	Fundamentação conceitual	2
3.1	FPGA, microcontrolador e ASIC	2
3.2	Papel das LUTs	2
4	Especificação funcional	2
5	Modelagem lógica da verificação do código	3
5.1	Código válido	3
5.2	Tabela verdade	3
5.3	Expressão booleana	4
5.4	Simplificação	4
6	Arquitetura do circuito	4
7	Decisões de projeto adotadas	5
8	Implementação em Verilog HDL	5
8.1	Estrutura lógica	5
8.2	Mapeamento físico na Tang Nano 9K	5
8.2.1	Recursos onboard	5
8.2.2	Botões externos do código	6
8.2.3	LEDs externos de apoio visual	6
8.3	Relação entre o enunciado e a montagem final	6
9	Simulação e validação	6
9.1	Cenários cobertos	6
9.2	Interpretação das formas de onda	7
9.3	Evidências de simulação	7
10	Implementação em FPGA e validação prática	10
10.1	Fluxo no Gowin	10
10.2	Evidências do fluxo de implementação	10
10.3	Evidências físicas do hardware	12
11	Vídeo demonstrativo	15
12	Conclusão	15

1 Introdução

Ao longo da disciplina, foram trabalhados conceitos de lógica booleana, circuitos combinacionais, circuitos sequenciais, descrição em Verilog HDL, testbench, análise de waveforms e implementação em FPGA. Como aplicação integradora desses conceitos, foi desenvolvido um sistema digital de controle de acesso capaz de validar um código de 4 bits, registrar tentativas e indicar os resultados por meio de LEDs.

O enunciado original previa a utilização da Tang Nano 4K e de switches para representar o código. Nesta implementação, o desenvolvimento físico foi realizado na Tang Nano 9K, com autorização prévia do professor, e os switches foram substituídos por botões externos montados em protoboard. Para tornar a operação prática no hardware, os botões dos bits do código foram tratados em modo toggle no wrapper da placa, sem alterar a lógica funcional exigida para a validação do código.

2 Objetivo do projeto

O objetivo do projeto foi implementar um sistema digital que:

- receba um código de 4 bits;
- compare esse código com o valor válido 1011;
- indique acesso autorizado ou negado;
- registre cada tentativa realizada;
- mantenha um contador síncrono de 3 bits para as tentativas;
- permita o reset completo do sistema;
- seja validado por simulação e por execução na FPGA.

3 Fundamentação conceitual

3.1 FPGA, microcontrolador e ASIC

Uma FPGA foi escolhida por ser uma plataforma reconfigurável e apropriada para o desenvolvimento e a validação rápida de sistemas digitais. Em comparação com um microcontrolador, a FPGA permite implementar diretamente a lógica em hardware, com paralelismo natural entre blocos combinacionais e sequenciais. Em comparação com um ASIC, a FPGA possui custo inicial muito menor e flexibilidade muito maior, o que a torna ideal para ensino, prototipagem e validação de arquitetura.

3.2 Papel das LUTs

Nas FPGAs, funções booleanas são implementadas internamente por LUTs (Look-Up Tables). Conceitualmente, uma LUT armazena a tabela verdade de uma função lógica e produz a saída correspondente para cada combinação de entrada. No contexto deste projeto, a função de validação do código pode ser entendida como uma função booleana de quatro variáveis, mapeável para os recursos lógicos internos da FPGA.

4 Especificação funcional

O sistema foi projetado para atender aos seguintes comportamentos:

- o código válido é 1011;
- quando o botão de confirmação é pressionado, o sistema verifica o código presente nas entradas;
- se o código estiver correto, o LED de acesso autorizado deve acender;
- se o código estiver incorreto, o LED de acesso negado deve acender;
- toda confirmação deve registrar uma tentativa;
- o contador de tentativas deve ser síncrono e possuir 3 bits;
- o botão de reset deve zerar o contador e apagar os LEDs de status.

Na adaptação física adotada:

- os quatro bits do código foram implementados com botões externos no protoboard;
- os botões onboard da Tang Nano 9K foram usados para confirmação e reset;
- os LEDs onboard da placa ficaram responsáveis pelos indicadores principais do enunciado;
- quatro LEDs externos adicionais foram incluídos apenas como apoio visual para mostrar o estado armazenado dos bits do código.

5 Modelagem lógica da verificação do código

5.1 Código válido

O código de acesso válido é 1011, correspondente à combinação:

- B3 = 1
- B2 = 0
- B1 = 1
- B0 = 1

5.2 Tabela verdade

B3	B2	B1	B0	F
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	0
0	1	0	0	0
0	1	0	1	0
0	1	1	0	0
0	1	1	1	0
1	0	0	0	0
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	0
1	1	0	1	0
1	1	1	0	0
1	1	1	1	0

5.3 Expressão booleana

Em forma canônica:

$$F(B3, B2, B1, B0) = \Sigma m(11)$$

Como existe apenas um mintermo válido, a expressão direta já é:

$$F = B3 \cdot \sim B2 \cdot B1 \cdot B0$$

5.4 Simplificação

A simplificação por álgebra booleana ou por mapa de Karnaugh leva ao mesmo resultado, pois existe apenas uma combinação válida. Não há agrupamentos adicionais possíveis.

Mapa de Karnaugh:

B3B2	B1B0	00	01	11	10
00		0	0	0	0
01		0	0	0	0
11		0	0	0	0
10		0	0	1	0

Portanto, a lógica combinacional implementada para a verificação do código permaneceu:

$$F = B3 \cdot \sim B2 \cdot B1 \cdot B0$$

6 Arquitetura do circuito

O projeto foi organizado em módulos com responsabilidades bem definidas, mantendo separação clara entre lógica combinacional e lógica sequencial.

Arquitetura modular adotada:

- `code_validator.v` Módulo puramente combinacional responsável por verificar se o vetor de 4 bits corresponde ao código 1011.
- `access_control.v` Módulo principal do sistema. Realiza a sincronização das entradas, detecta o pulso de confirmação, atualiza o contador de tentativas e registra os LEDs de autorizado e negado.
- `button_debouncer.v` Módulo sequencial usado na integração com a placa para filtrar o ruído mecânico dos botões físicos.
- `access_control_tang_nano_9k_top.v` Wrapper específico da Tang Nano 9K. Faz a adaptação elétrica e funcional dos sinais da placa, incluindo tratamento de níveis ativos, debounce dos botões e conversão dos quatro botões externos do código para modo toggle.

Fluxo de funcionamento:

1. Os quatro bits do código chegam ao sistema.
2. O módulo combinacional valida se o código é 1011.
3. Quando ocorre uma confirmação, a tentativa é registrada.

4. O contador síncrono de 3 bits é incrementado.
5. As saídas de autorizado e negado refletem o resultado da última tentativa.
6. O reset zera contador e LEDs de status.

7 Decisões de projeto adotadas

As principais decisões de projeto foram:

- manter a verificação do código em módulo combinacional separado, aderente à expressão booleana obtida;
- implementar o contador de tentativas como registrador síncrono de 3 bits;
- sincronizar os sinais de entrada antes do uso na lógica principal;
- detectar o evento de confirmação por pulso de borda, evitando múltiplos incrementos por uma mesma pressão prolongada;
- isolar a dependência da placa em um wrapper específico da Tang Nano 9K;
- usar debounce para os botões físicos;
- adaptar os quatro botões do código para operação em modo toggle na placa, devido à substituição dos switches por botões externos.

Também foi adotado um conjunto de LEDs extras no protoboard para facilitar a visualização dos bits do código durante a demonstração. Esses LEDs não substituem os LEDs previstos no enunciado; eles apenas complementam a usabilidade durante a operação do sistema.

8 Implementação em Verilog HDL

8.1 Estrutura lógica

A implementação em Verilog foi dividida em duas partes:

- **parte lógica genérica**, independente da placa;
- **parte de integração física**, responsável pela associação entre sinais lógicos e recursos da Tang Nano 9K.

Essa separação torna o projeto mais organizado e facilita a reutilização da lógica principal em outras plataformas.

8.2 Mapeamento físico na Tang Nano 9K

8.2.1 Recursos onboard

Função lógica	Recurso físico	Pino FPGA
<code>sys_clk</code>	clock onboard	52
<code>btn_confirm_n</code>	botão onboard S1	3
<code>btn_reset_n</code>	botão onboard S2	4
<code>led[0]</code>	LED1 onboard	10
<code>led[1]</code>	LED2 onboard	11
<code>led[2]</code>	LED3 onboard	13
<code>led[3]</code>	LED4 onboard	14
<code>led[4]</code>	LED5 onboard	15

Função lógica	Recurso físico	Pino FPGA
1ed[5]	LED6 onboard	16

8.2.2 Botões externos do código

Bit do código	Sinal	Pino FPGA
bit 0	code_btn[0]	25
bit 1	code_btn[1]	26
bit 2	code_btn[2]	27
bit 3	code_btn[3]	28

8.2.3 LEDs externos de apoio visual

Bit exibido	Sinal	Pino FPGA
bit 0	code_led[0]	29
bit 1	code_led[1]	30
bit 2	code_led[2]	41
bit 3	code_led[3]	42

8.3 Relação entre o enunciado e a montagem final

Embora o enunciado cite a Tang Nano 4K e switches físicos, a lógica funcional permaneceu equivalente:

- os quatro botões externos substituem os quatro bits antes definidos por switches;
- o botão onboard S1 atua como confirmação;
- o botão onboard S2 atua como reset;
- os LEDs onboard representam as saídas obrigatórias do enunciado;
- os LEDs externos apenas mostram o estado armazenado dos bits do código.

9 Simulação e validação

Antes da implementação em hardware, o projeto foi validado por simulação com testbenches dedicados.

9.1 Cenários cobertos

Os testbenches verificaram:

- geração de clock;
- aplicação de reset;
- código correto;
- código incorreto;
- múltiplas tentativas;
- reset do sistema;
- comportamento toggle dos botões externos na versão de placa.

9.2 Interpretação das formas de onda

O conjunto de sinais analisado permitiu validar:

- a verificação do código;
- o funcionamento do contador;
- o comportamento das saídas;
- o registro da tentativa durante a confirmação;
- a adaptação dos botões externos para modo toggle.

O contador foi mantido em representação binária, conforme solicitado no enunciado:

- 1 tentativa = 001
- 2 tentativas = 010
- 3 tentativas = 011
- 4 tentativas = 100

9.3 Evidências de simulação

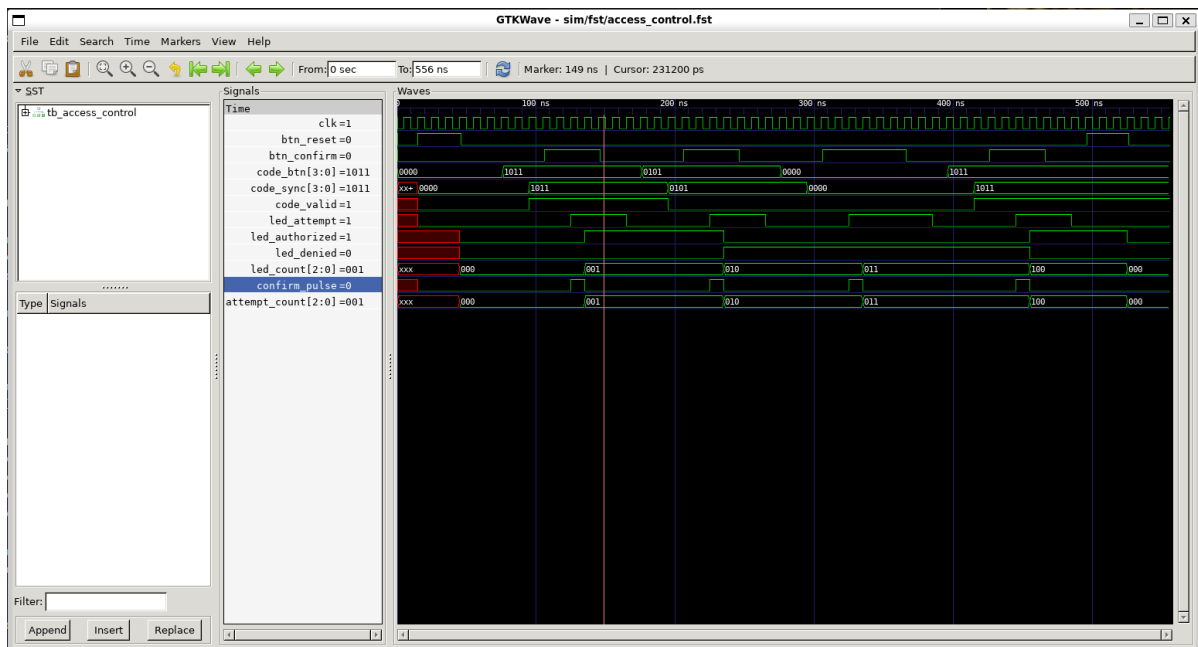


Figure 1: Validação do código correto 1011: o código sincronizado é reconhecido como válido, a confirmação gera a tentativa e o LED de acesso autorizado é ativado.

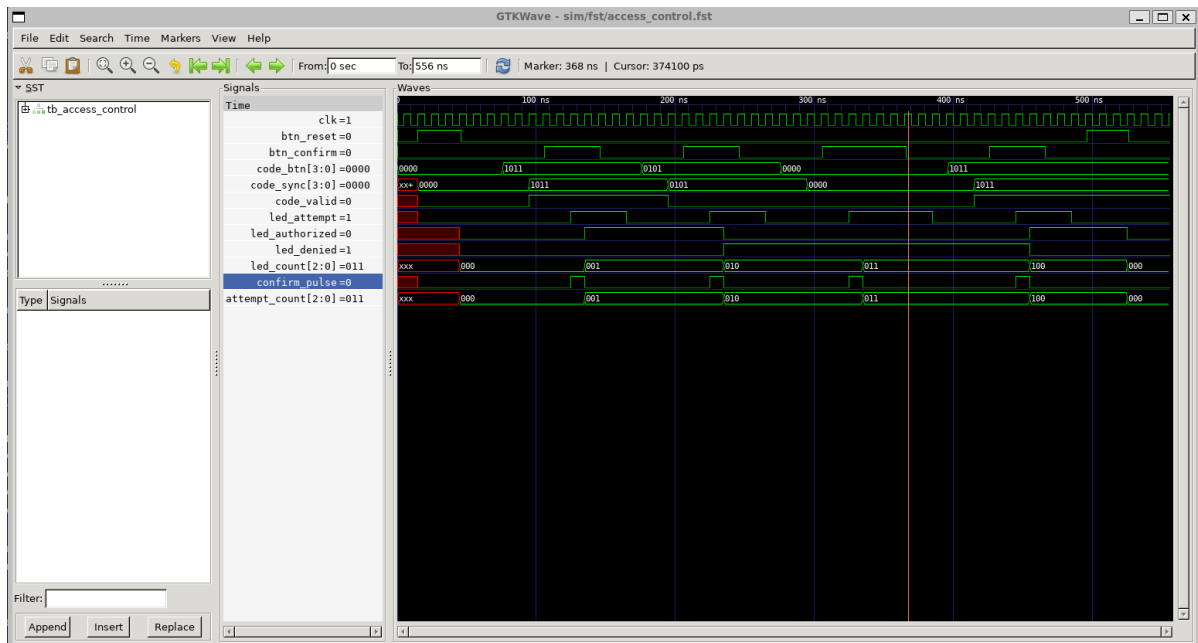


Figure 2: Validação de código incorreto: a confirmação mantém a lógica de tentativa, porém a saída de acesso negado é ativada e a de autorizado permanece inativa.

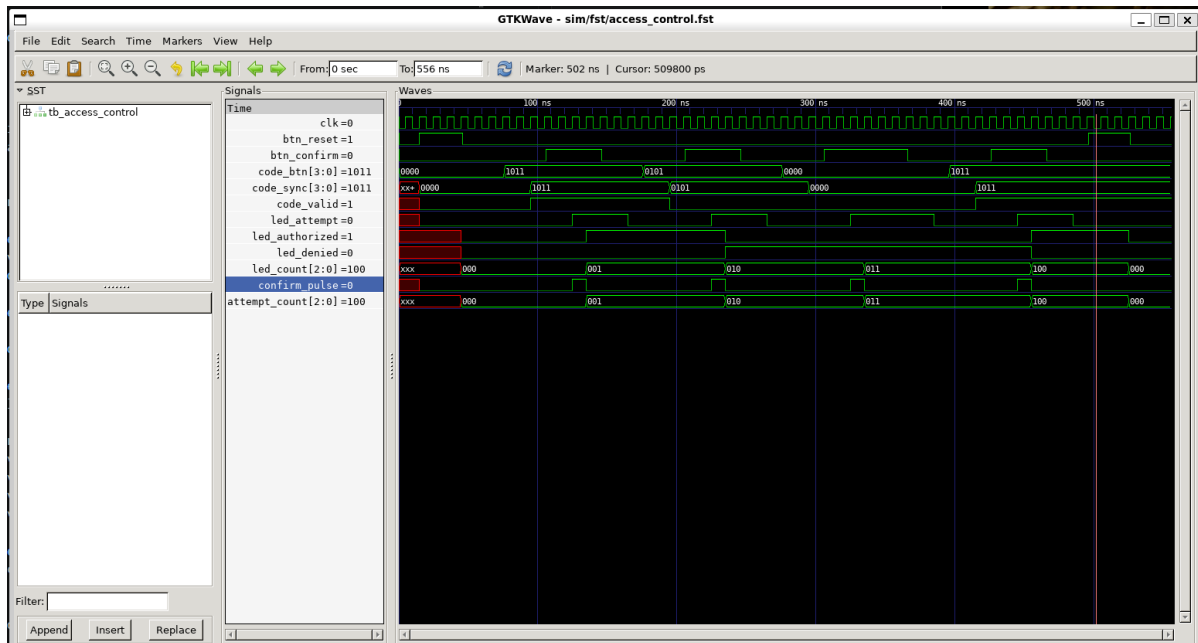


Figure 3: Funcionamento do contador de tentativas em binário: as transições 001, 010, 011 e 100 demonstram o incremento síncrono a cada confirmação.

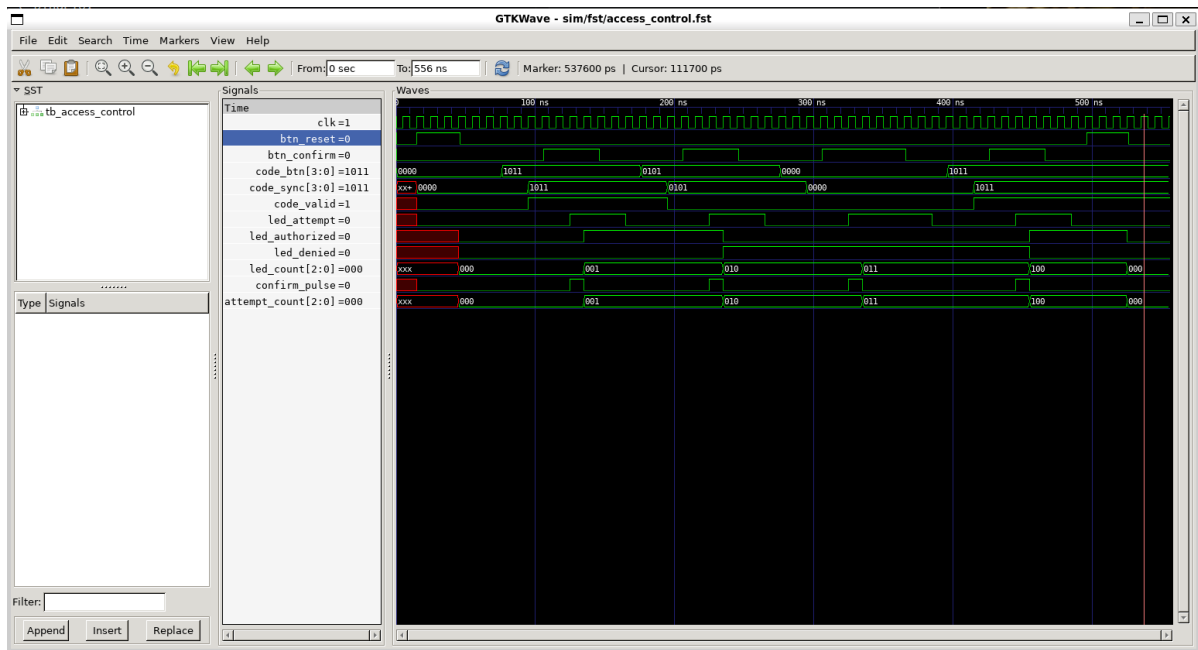


Figure 4: Reset do sistema: o contador retorna para 000 e as saídas de status são apagadas após o acionamento do reset.

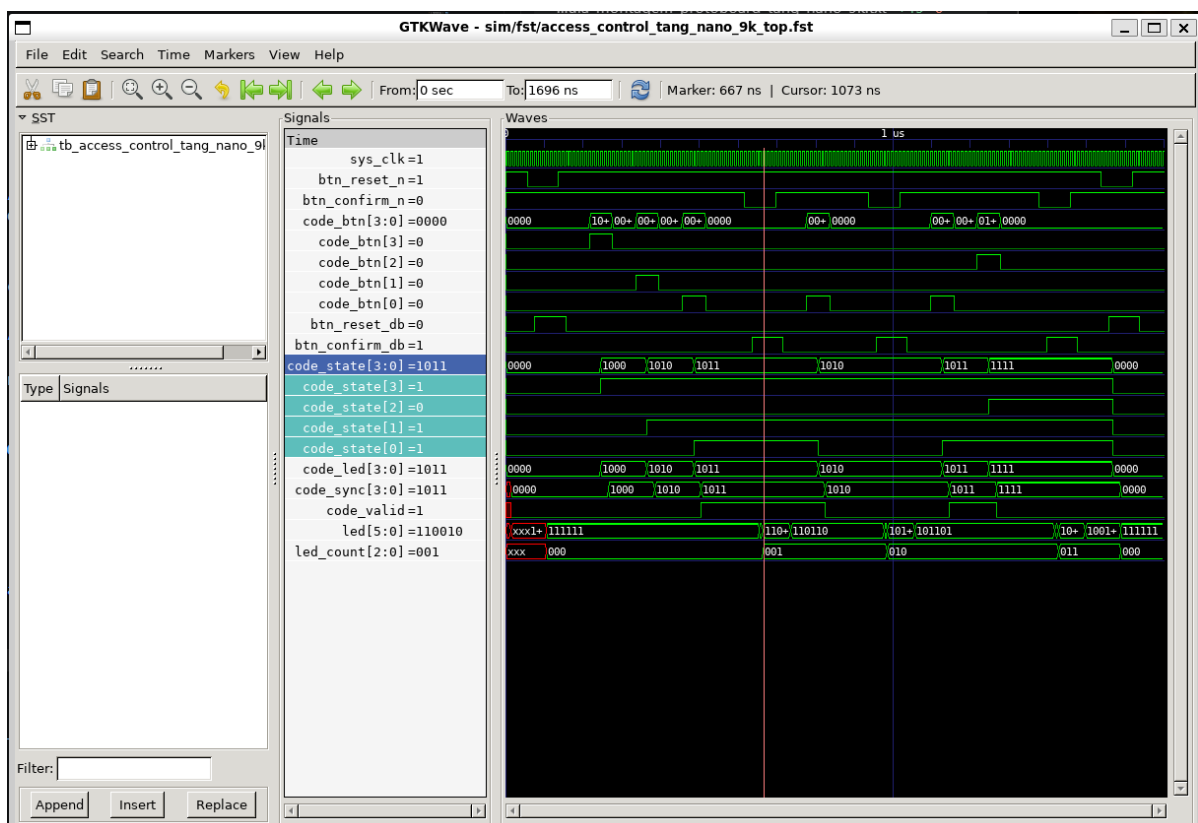


Figure 5: Waveform da integração com a Tang Nano 9K: toques curtos nos botões externos alteram o estado armazenado dos bits em modo toggle e os LEDs auxiliares acompanham esse estado.

10 Implementação em FPGA e validação prática

Após a validação por simulação, o projeto foi integrado ao fluxo do Gowin para síntese, place-and-route, geração de bitstream e gravação na FPGA.

10.1 Fluxo no Gowin

O projeto foi preparado para ser aberto diretamente no Gowin IDE por meio do arquivo `gowin/access_control_tang_nano_9k.gprj`, incluindo:

- top-level da placa;
- arquivo de constraints;
- restrição de clock;
- configuração básica do fluxo.

As etapas realizadas foram:

1. síntese;
2. place and route;
3. geração do bitstream;
4. programação da Tang Nano 9K.

10.2 Evidências do fluxo de implementação

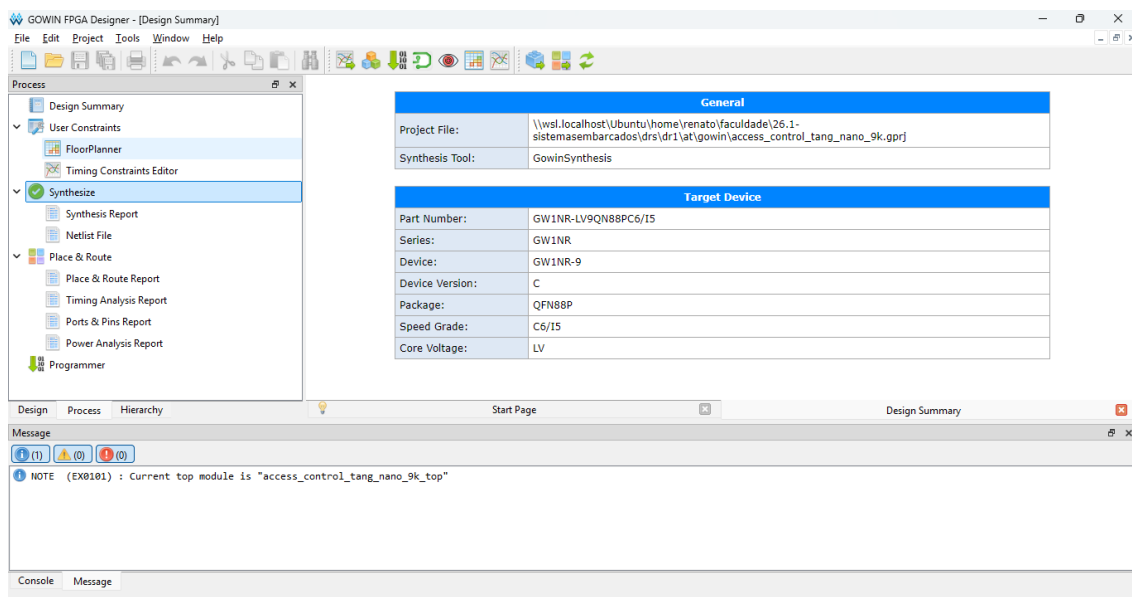


Figure 6: Síntese concluída com sucesso no Gowin, confirmando que a descrição em Verilog foi aceita e processada corretamente pelo fluxo de compilação.

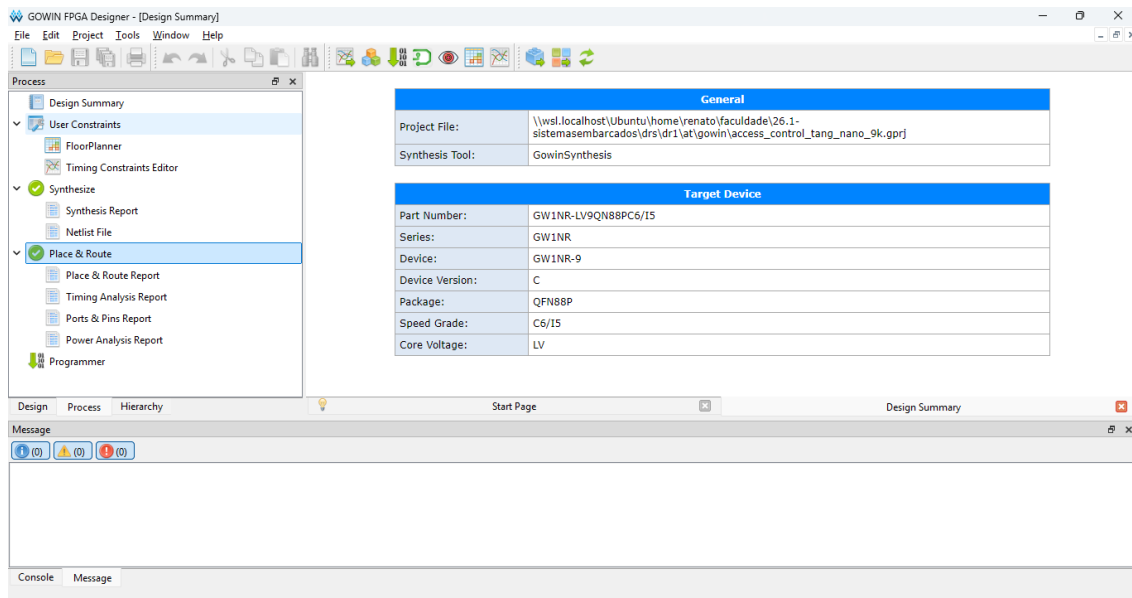


Figure 7: Place and route concluído com sucesso, indicando que o projeto foi mapeado para os recursos físicos da FPGA com a pinagem definida.

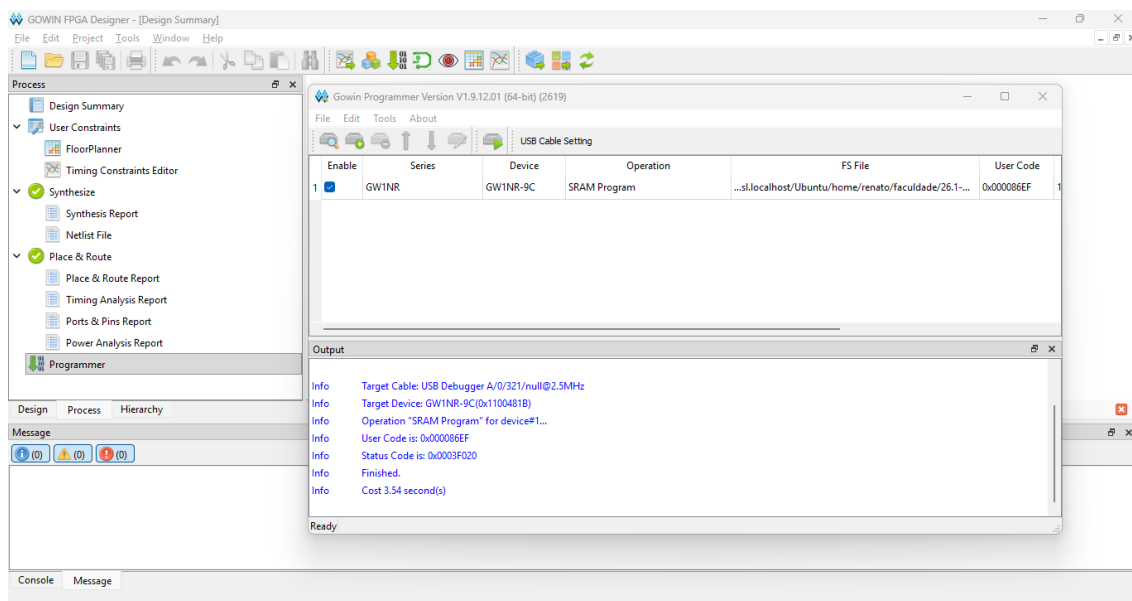


Figure 8: Programação da Tang Nano 9K concluída com sucesso no programador do Gowin.

10.3 Evidências físicas do hardware

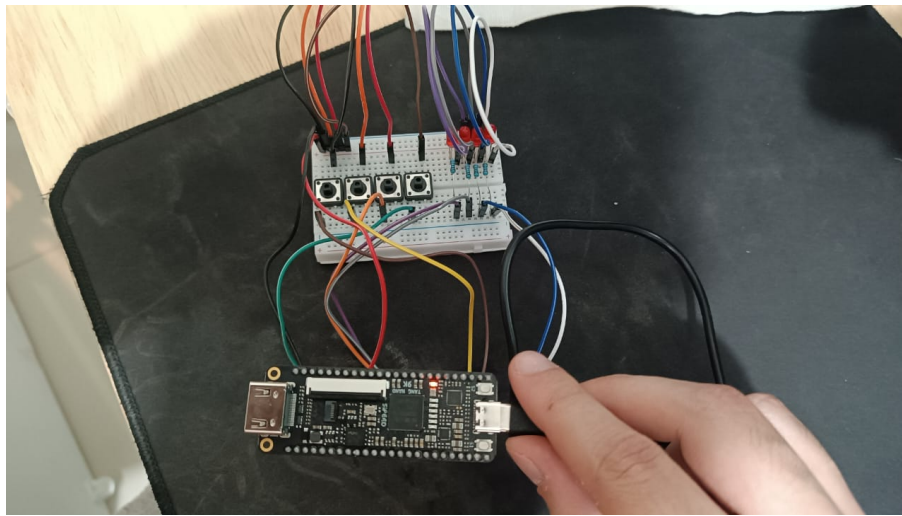


Figure 9: Montagem geral do sistema com Tang Nano 9K, protoboard, botões externos do código e LEDs auxiliares de visualização.

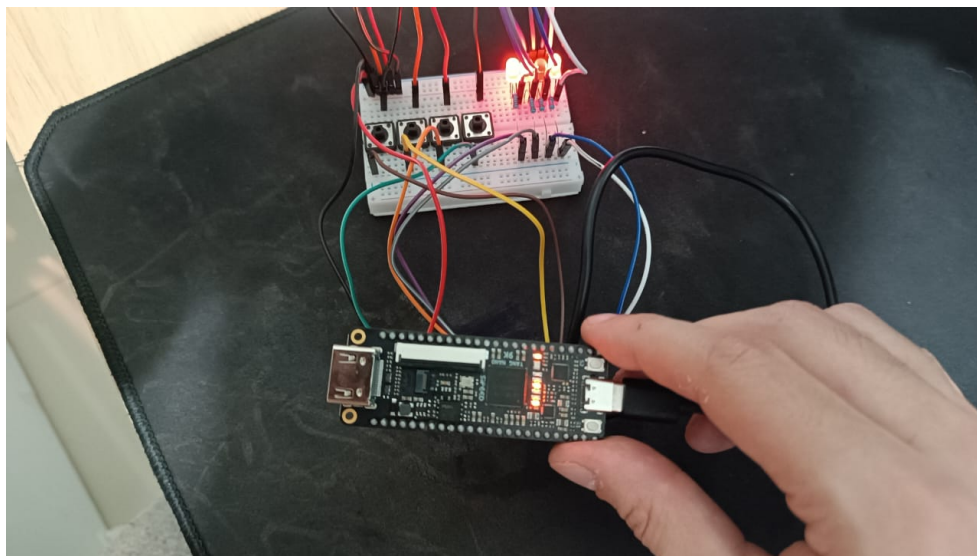


Figure 10: Demonstração do código correto 1011, com indicação visual dos bits armazenados e sinalização de acesso autorizado nos LEDs principais.

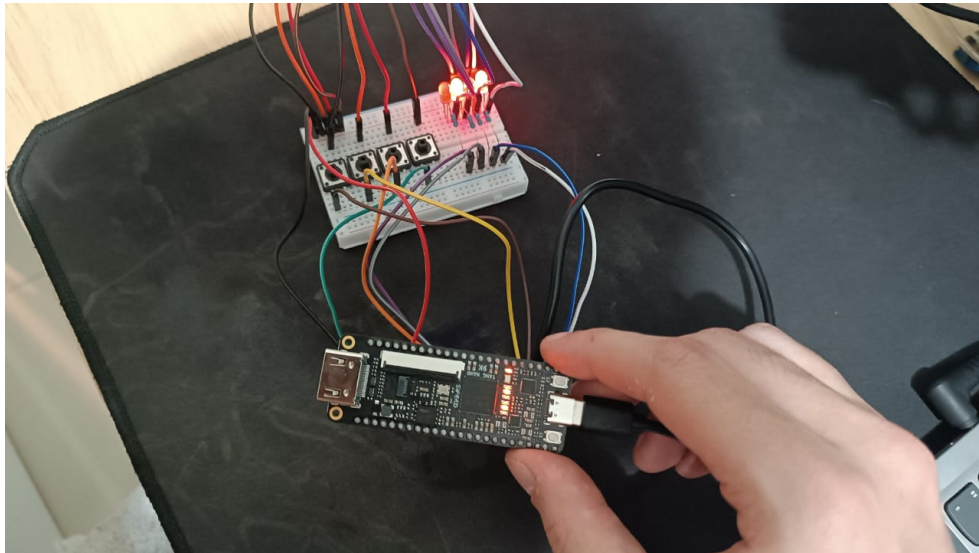


Figure 11: Demonstração de código incorreto, com resposta de acesso negado na placa.

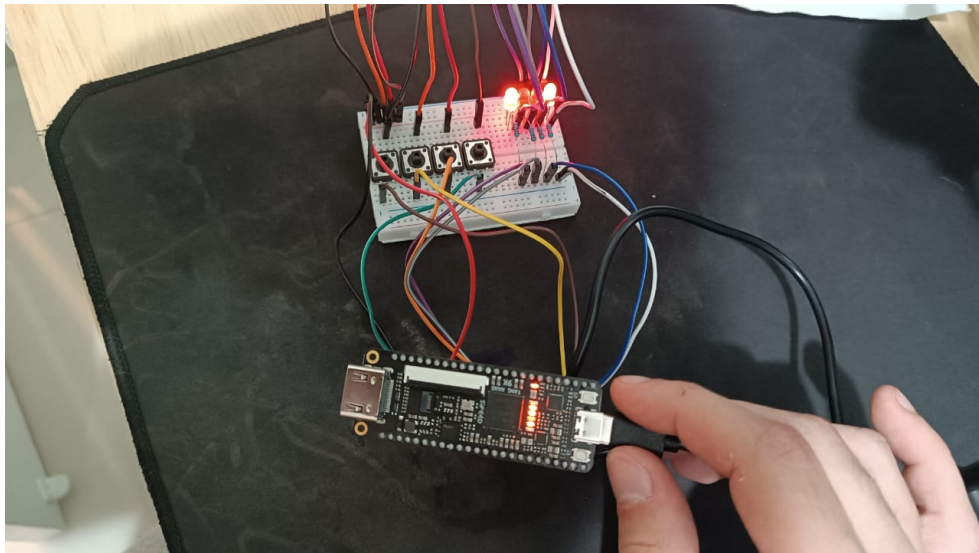


Figure 12: Registro de múltiplas tentativas com o contador binário exibido nos LEDs correspondentes da placa.

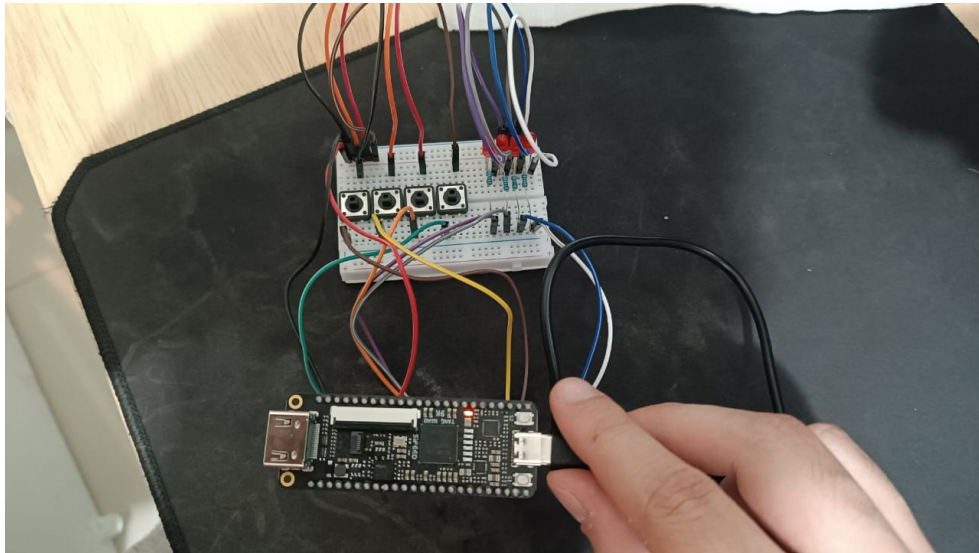


Figure 13: Reset do sistema, com retorno do contador ao zero e apagamento das saídas de status.

11 Vídeo demonstrativo

O funcionamento completo do sistema também foi registrado em vídeo, incluindo interação com os botões e resposta observada nos LEDs.

Link do vídeo demonstrativo:

https://drive.google.com/file/d/14vj8VbRwjhtH6EMj51q2ctrXvOo8D6Z/view?usp=drive_link

12 Conclusão

O projeto atendeu aos objetivos propostos para o sistema digital de controle de acesso. A lógica de verificação do código foi modelada formalmente, implementada em Verilog HDL e validada por simulação. O sistema foi então integrado à Tang Nano 9K, sintetizado, programado e testado fisicamente com sucesso.

Os resultados obtidos demonstraram:

- funcionamento correto da verificação do código 1011;
- distinção clara entre acesso autorizado e acesso negado;
- registro de tentativa a cada confirmação;
- incremento correto do contador síncrono de 3 bits;
- operação adequada do reset;
- correspondência entre simulação e execução em hardware.

Além de atender aos requisitos centrais do enunciado, o projeto incorporou adaptações práticas importantes para o contexto disponível de hardware, especialmente a substituição dos switches por botões externos e a utilização da Tang Nano 9K com aprovação do professor. Essas adaptações mantiveram a equivalência funcional do sistema e permitiram uma demonstração física clara, estável e aderente ao objetivo didático da atividade.